

Trabajo final para subida de nota y Evaluación Estructuras Dinámicas:

Gestión de una Liga de E-sports.

1. Introducción.

A lo largo del curso se han trabajado distintos contenidos fundamentales de programación en Java: programación estructurada, programación modular, programación orientada a objetos, arrays, matrices, excepciones, clases abstractas, interfaces y estructuras dinámicas.

Como actividad final de integración, se propone el desarrollo de una aplicación completa en Java en la que se pongan en práctica de forma conjunta y coherente todos esos contenidos.

El trabajo no consistirá en resolver ejercicios aislados, sino en diseñar e implementar un sistema funcional con varias entidades relacionadas entre sí, reglas de negocio, control de errores y uso justificado de distintas estructuras de datos.

Al haber sido vuestro profesor en este curso, estoy orgulloso de vuestro desempeño y confío plenamente en vuestras capacidades. Podéis enfrentaros a este trabajo.

2. Objetivo del trabajo.

Desarrollar una aplicación en Java que permita gestionar de forma integral una liga de e-sports, incluyendo personas de la liga, equipos, plantillas, calendario, partidos, incidencias, sanciones, clasificación e historial de operaciones.

El programa deberá reflejar de forma clara el dominio de los contenidos trabajados durante el curso y estar estructurado de forma modular, legible y robusta. No se permite hacer un programa con todo en main.

El trabajo debe reflejar claramente que el alumnado domina los contenidos del curso.

3. Duración del proyecto.

- El proyecto comienza el miércoles 29 de abril.
- El proyecto se entregará el viernes 15 de mayo.

Se recomienda no dejar el proyecto, completamente, a manos de la Inteligencia Artificial y tampoco se recomienda dejarlo para el último día. Lo lógico es utilizar las horas de clase para avanzar en él.

4. Descripción general.

La empresa organizadora **1GSY** desea disponer de una aplicación que permita administrar su competición de e-sports.

En la liga participan varios equipos. Cada equipo cuenta con un entrenador y una plantilla formada por jugadores titulares y suplentes. A lo largo de la temporada se generan jornadas, se programan partidos, se registran resultados, se aplican sanciones, se actualizan estadísticas y se obtiene una clasificación final.

La aplicación deberá ejecutarse completamente por consola, mediante menús, submenús y validaciones adecuadas.

5. Funcionalidades obligatorias

- 5.1. Gestión de personas de la liga.

El sistema deberá permitir:

- dar de alta personas de la liga
- listar todas las personas registradas
- buscar una persona por identificador
- modificar determinados datos
- eliminar personas cuando proceda

Deberá existir una clase abstracta llamada, PersonaLiga.

Atributos mínimos de PersonaLiga

- identificador
- nombre
- nickname
- edad
- salario base

Métodos mínimos de PersonaLiga

- constructores
- getters y setters
- mostrarResumen()
- calcularCosteMensual()
- toString()

calcularCosteMensual() deberá ser un método abstracto o un método sobrescrito de forma distinta por las subclases.

Subclases obligatorias:

1. Jugador

Atributos mínimos:

- rol
- nivel mecánico
- nivel estratégico
- partidas jugadas
- mvp totales
- sancionado o no

2. Entrenador

Atributos mínimos:

- experiencia
- especialidad
- victorias totales

● 5.2. Interfaz obligatoria: Entrenable

Métodos:

- void entrenar()
- double calcularRendimiento()

La implementarán al menos los jugadores.

● 5.3. Gestión de equipos.

Cada equipo deberá tener, como mínimo:

- nombre
- ciudad
- entrenador
- presupuesto
- número de victorias
- número de derrotas
- puntos a favor
- puntos en contra

El sistema deberá permitir:

- crear equipos

- asignar entrenador a un equipo
- fichar jugadores
- mostrar plantilla completa
- sustituir titulares por suplentes
- eliminar jugadores
- mostrar coste total del equipo
- impedir nombres de equipo repetidos

- 5.4. Uso obligatorio de arrays.

Cada equipo deberá tener un array fijo de jugadores titulares. Ejemplo orientativo:

```
Jugador[] titulares = new Jugador[5];
```

El array deberá utilizarse con sentido real y ser parte central del diseño.

No se considerará válido crear un array sin uso relevante.

- 5.5. Uso obligatorio de listas.

Cada equipo deberá tener además una lista dinámica de jugadores suplentes. Ejemplo orientativo:

```
ArrayList<Jugador> suplentes;
```

La lista deberá permitir:

- añadir suplentes
- eliminar suplentes
- recorrer la colección
- promover suplentes a titulares cuando proceda

- 5.6. Roles de jugadores.

Cada jugador deberá pertenecer a un rol:

TOP

JUNGLE

MID

ADC

SUPPORT

No puede haber dos titulares con el mismo rol dentro del mismo equipo.

Si se intenta incumplir esta regla, deberá controlarse mediante validación y excepción.

- 5.7. Estructura dinámica.

Debe utilizarse alguna estructura dinámica para controlar duplicados de forma real.
Por ejemplo:

- IDs de personas ya registradas
- nombres de equipos ya existentes
- identificadores de partidos ya creados

- 5.8. Calendario y jornadas.

La liga deberá disponer de una matriz bidimensional con utilidad real.

Algunas posibilidades válidas son:

- calendario de jornadas
- matriz de enfrentamientos
- puntos por jornada y equipo
- resultados cruzados entre equipos

El sistema deberá permitir, como mínimo:

1. generar o almacenar un calendario
2. mostrar el calendario completo
3. consultar información de una jornada
4. consultar resultados o estadísticas por jornada

La matriz debe estar integrada en el funcionamiento del programa y no ser un simple elemento decorativo.

- 5.9. Cola.

Los partidos que aún no se hayan disputado deberán almacenarse en una cola.

El sistema deberá permitir:

1. encolar partidos pendientes
2. mostrar el siguiente partido pendiente
3. disputar el siguiente partido
4. mostrar todos los partidos pendientes
5. vaciar la cola cuando proceda

Debe apreciarse claramente el comportamiento FIFO.

- 5.10. Pila de historial de acciones.

El programa deberá disponer de una **pila** para almacenar las últimas acciones realizadas por el usuario o administrador.

Ejemplo orientativo:

- alta de persona
- creación de equipo
- fichaje
- sanción
- registro de partido

El sistema deberá permitir:

1. registrar acciones en la pila
2. mostrar la última acción realizada
3. mostrar el historial almacenado
4. deshacer, al menos de forma parcial, alguna operación sencilla

Debe apreciarse claramente el comportamiento LIFO.

- 5.11. Registro de partidos.

Deberá existir una clase Partido con, al menos, la siguiente información:

- identificador del partido
- jornada
- equipo local
- equipo visitante
- puntuación del local
- puntuación del visitante
- jugador MVP
- estado de disputado o no disputado

El sistema deberá permitir:

1. crear partidos
2. registrar el resultado de un partido
3. calcular el ganador
4. actualizar estadísticas de los equipos
5. actualizar estadísticas individuales cuando proceda
6. impedir registrar dos veces un mismo partido
7. impedir partidos no válidos, como un equipo contra sí mismo

- 5.12. Incidencias y sanciones.

Deberá existir una clase Incidencia.

Las incidencias podrán estar relacionadas con:

- sanciones a jugadores
- expulsiones
- errores técnicos
- partidos aplazados
- cualquier otra situación definida con coherencia

Las incidencias se almacenarán en una lista dinámica.

El sistema deberá permitir:

1. registrar incidencias
 2. listar incidencias
 3. buscar incidencias por equipo o por jugador
 4. aplicar sanciones
 5. impedir que un jugador sancionado entre en una convocatoria válida
- 5.13. Convocatorias para partidos.

Para disputar un partido, cada equipo deberá generar una convocatoria válida.

Reglas mínimas

- deben participar exactamente 5 titulares
- no puede jugar un jugador sancionado
- no puede haber dos titulares con el mismo rol
- si falta un titular, podrá incorporarse un suplente compatible si la lógica diseñada lo permite

Esta funcionalidad debe estar implementada mediante métodos y validaciones específicas.

- 5.14. Clasificación final y estadísticas.

El programa deberá ser capaz de generar y mostrar:

Estadísticas de jugadores

- partidas jugadas
- MVP obtenidos
- rendimiento medio
- estado de sanción
- coste o valor calculado si así se diseña

Estadísticas de equipos

- victorias
- derrotas
- puntos a favor
- puntos en contra
- diferencia de puntos
- coste total de plantilla

Clasificación general

La clasificación deberá mostrarse ordenada siguiendo un criterio coherente, por ejemplo:

1. número de victorias
2. diferencia de puntos
3. nombre del equipo

La clasificación debe calcularse a partir de los datos almacenados y no introducirse manualmente.

6. Excepciones obligatorias

El proyecto deberá incluir control de errores mediante try-catch cuando proceda y deberá crear al menos dos excepciones personalizadas.

Ejemplos válidos:

PersonaDuplicadaException

EquipoDuplicadoException

RolNoDisponibleException

PartidoInvalidoException

JugadorSancionadoException

Las excepciones deberán lanzarse y gestionarse en el programa. No bastará con declararlas sin uso.

7. Estructura mínima recomendada del proyecto

El proyecto deberá incluir, como mínimo, clases similares a las siguientes:

- Main
- Liga
- Equipo
- Partido
- Incidencia
- PersonaLiga
- Jugador
- Entrenador
- Entrenable
- Excepciones personalizadas

El alumnado podrá ampliar esta estructura con nuevas clases si lo considera oportuno.

8. Menú mínimo orientativo

La aplicación deberá contar con un menú general funcional. A modo orientativo, puede organizarse de la siguiente forma:

1. Gestionar personas de la liga
2. Gestionar equipos
3. Gestionar fichajes y plantillas
4. Gestionar calendario
5. Gestionar cola de partidos
6. Registrar partidos jugados
7. Gestionar incidencias y sanciones
8. Mostrar clasificación
9. Mostrar estadísticas
10. Mostrar historial de acciones
11. Deshacer última acción
12. Salir

El alumnado podrá reorganizar submenús y opciones, siempre que se mantengan todas las funcionalidades obligatorias.

9. Entregables.

- Carpeta zip comprimida del proyecto. El nombre de la carpeta tendrá el siguiente formato:

Primerapellido_Segundoapellido_Nombre_Proyecto_Final

- Memoria técnica en PDF de extensión orientativa entre 6 y 10 páginas. Deberá incluir:

- portada
- descripción general del proyecto
- explicación del diseño de clases
- justificación del uso de estructuras dinámicas
- explicación de las excepciones

El nombre de la memoria tendrá el siguiente formato:

Primerapellido_Segundoapellido_Nombre_Memoria_Proyecto_Final

- El primer día tras la entrega, se realizarán algunas preguntas sobre los códigos de cada uno, para así poder validar el proceso de aprendizaje seguido por el estudiante y para corroborar que ha comprendido su trabajo.